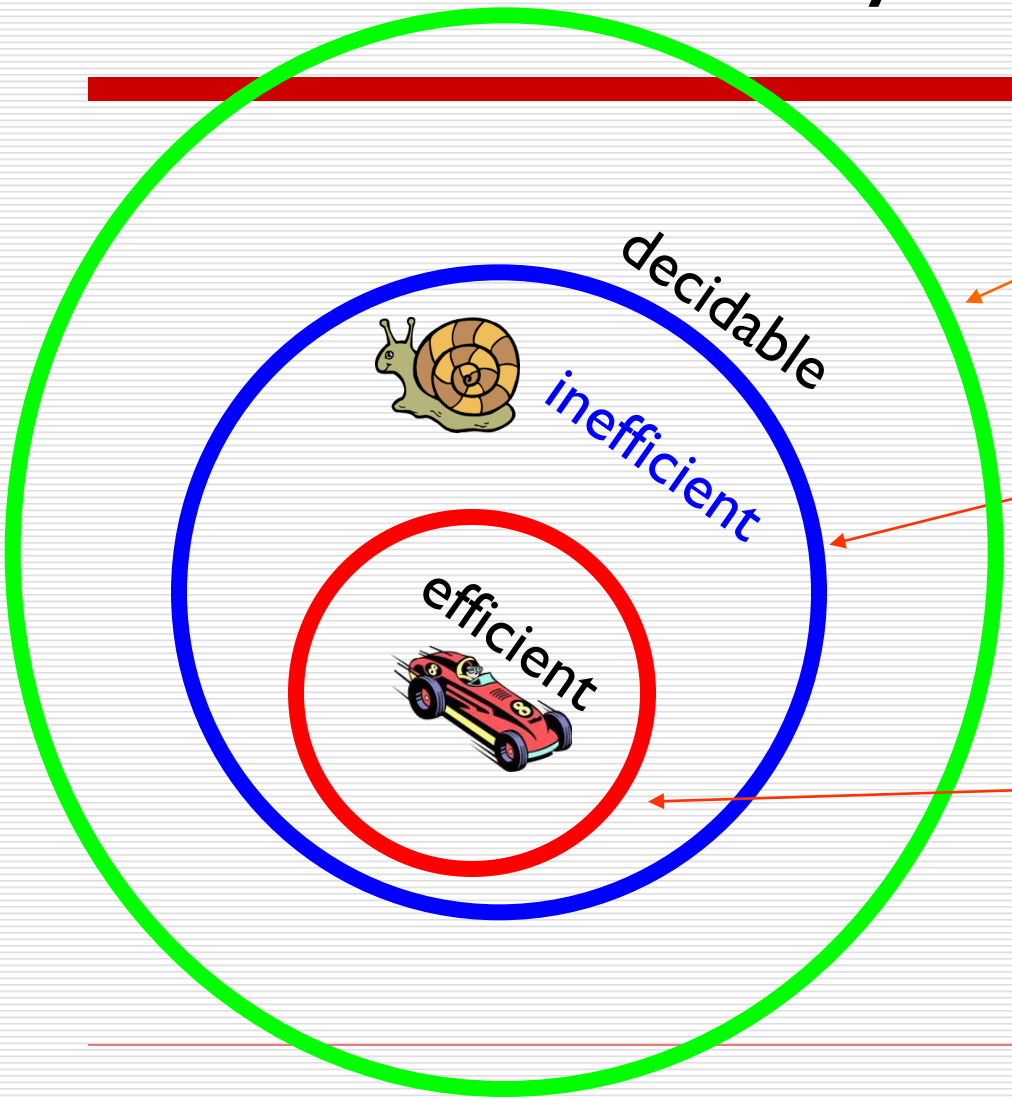
Lecture 2 Finite Automata

- Deterministic finite automata,
- nondeterministic finite automata
- their equivalence and
- minimization

Classical Theory



Algorithm

Decidability

Efficiency (P vs. NP)

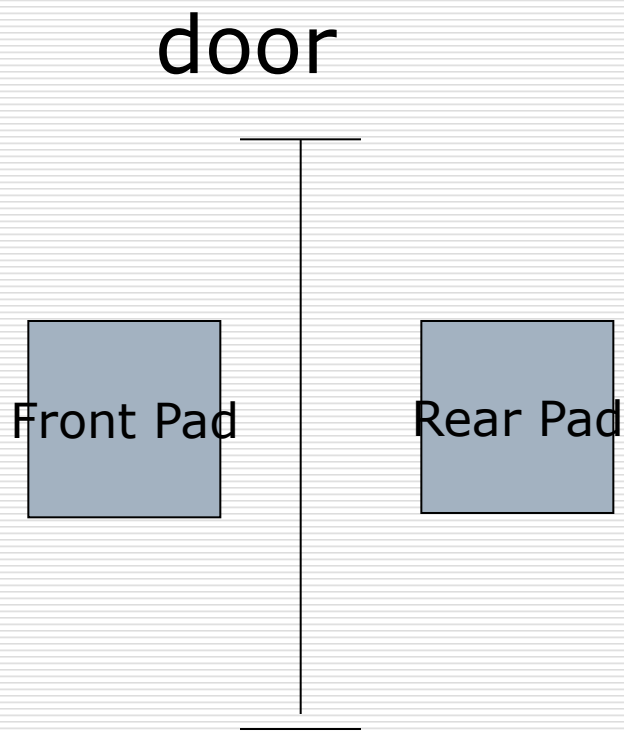
Structural Induction

Definition: An expression is defined inductively as follows:

1. Any number or letter is an expression
2. If E and F are expressions, so are E+F. E*F and (E).

Theorem: Every expression has an equal number of left and right parentheses.

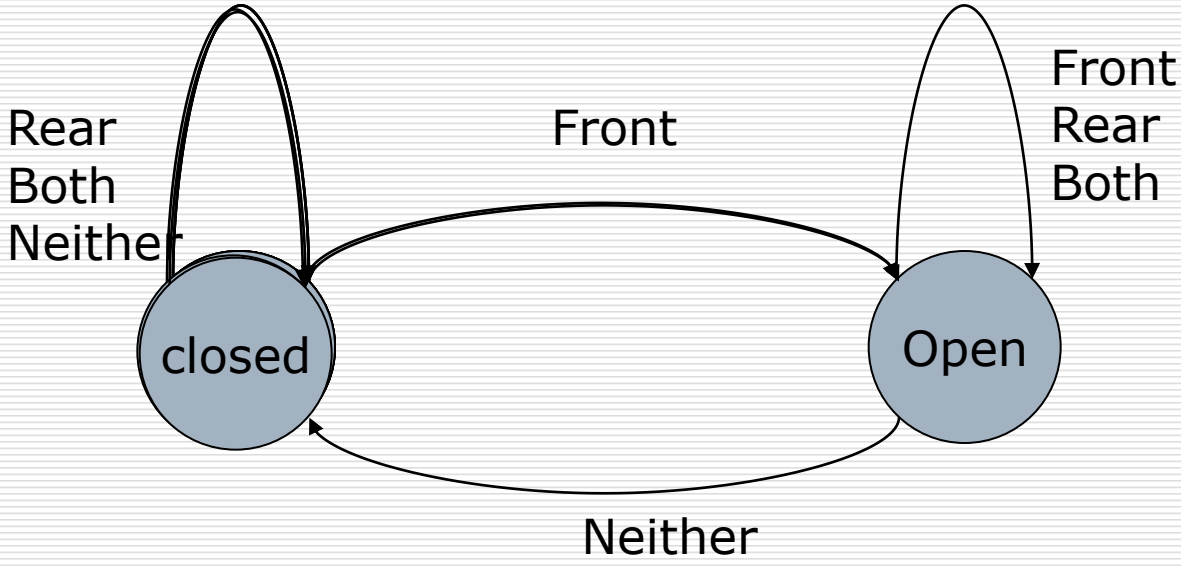
Example: Automatic door



How do you automatize ATM?

State Diagram

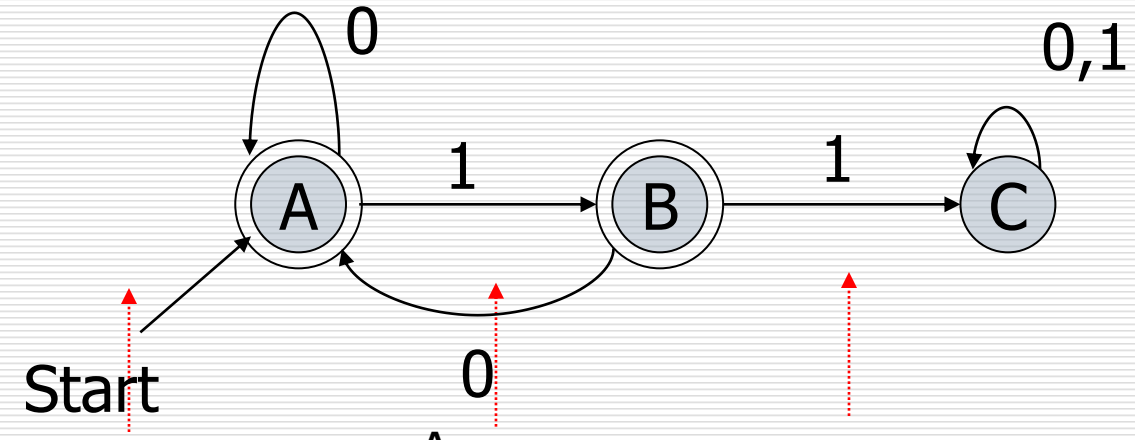
Automatic Door as an automaton



	Neither	Front	Rear	Both
Closed	Closed	Open	Closed	Closed
open	Open	Closed	Open	Open

Example: A Simple Automata

Accepts all strings without two consecutive 1's.



Previous string OK, does not end in 1.

Previous String OK, ends in a single 1.

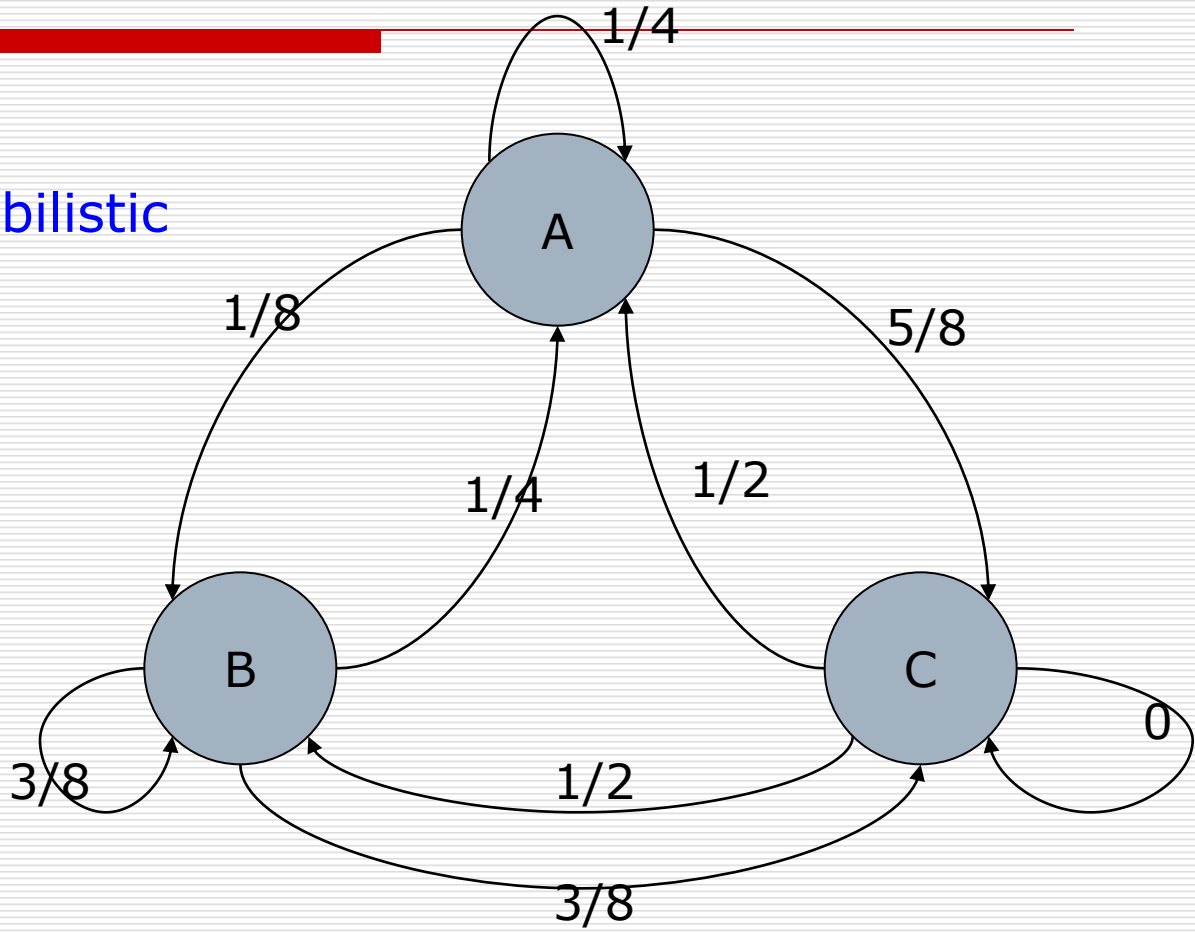
Consecutive 1's have been seen.

A Mathematical "Cousin"

Markov Chain:

How to define a probabilistic automaton

	A	B	C
A	1/4	1/8	5/8
B	1/4	3/8	3/8
C	1/2	1/2	0



Formal Definition

A deterministic finite automaton (DFA) is a 5 tuple $(Q, \Sigma, \delta, q_0, F)$

1. Q is a finite set called the states;
2. Σ is a finite set called the alphabet;
3. $\delta : Q \times \Sigma \to Q$ is the transition function;
4. q_0 is the start state, and
5. $F \subseteq Q$ is the set of accepting states.

Formal Definition of Computation

- A DFA M accepts input w if w can be written as $w = w_1 w_2 \cdots w_m$ where $w_i \in \Sigma$ and sequences of states $r_0, \cdots, r_m \in Q$ exist that satisfy the following conditions:

1. $r_0 = q_0$
2. For $i = 0, \cdots, m-1$, we have $\delta(r_i, w_{i+1}) = r_{i+1}$
3. rm is an accept state.

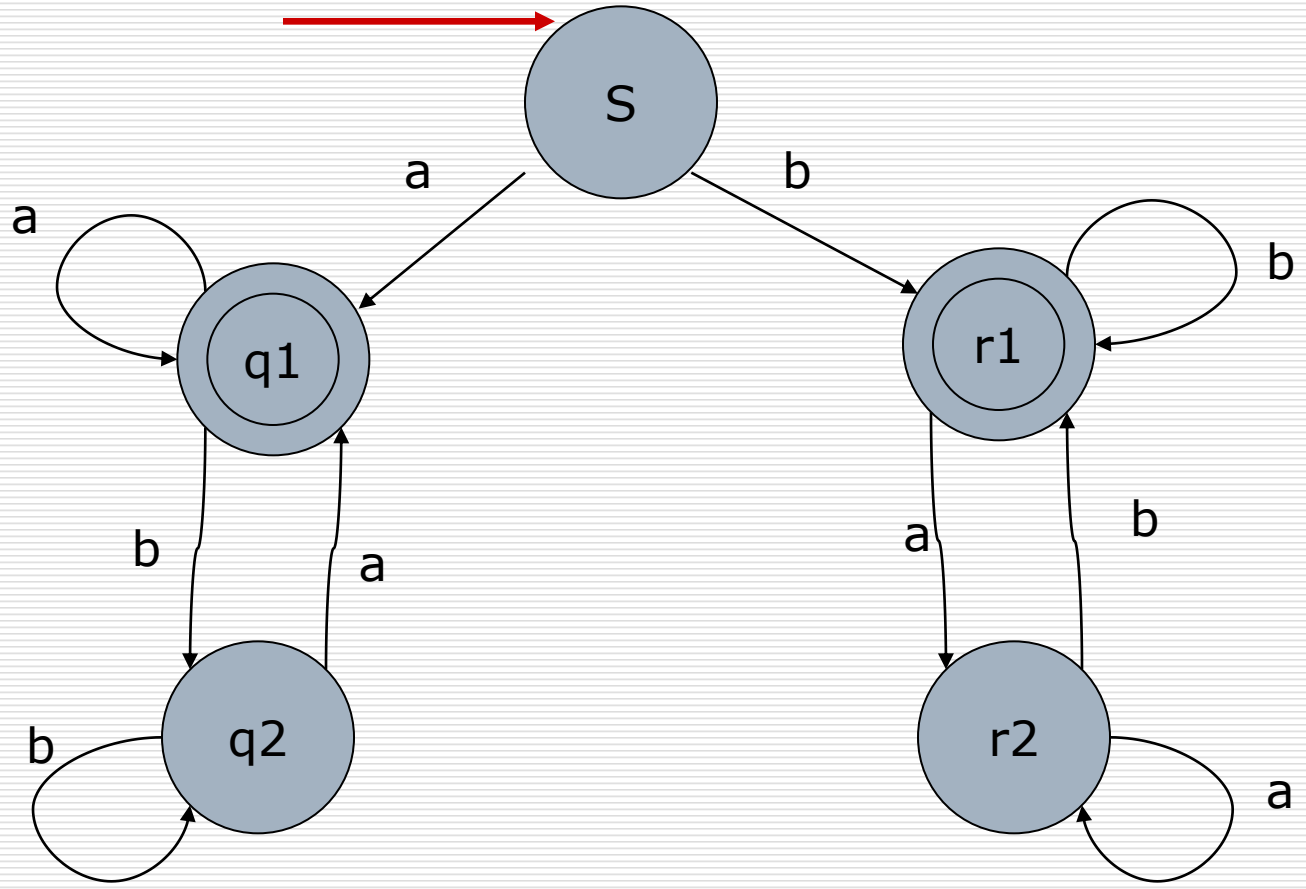
Computations,Derivations and Proofs

Languages recognized by FA

- If A is the set of all strings that machine M accepts, then A is called the language of M and write L(M)=A. We say that M recognizes A or M accepts A.
- Two machines are equivalent if they recognize the same languages.

If machine M accepts no strings, then L(M) is the empty language.

Example: a 5-state automaton



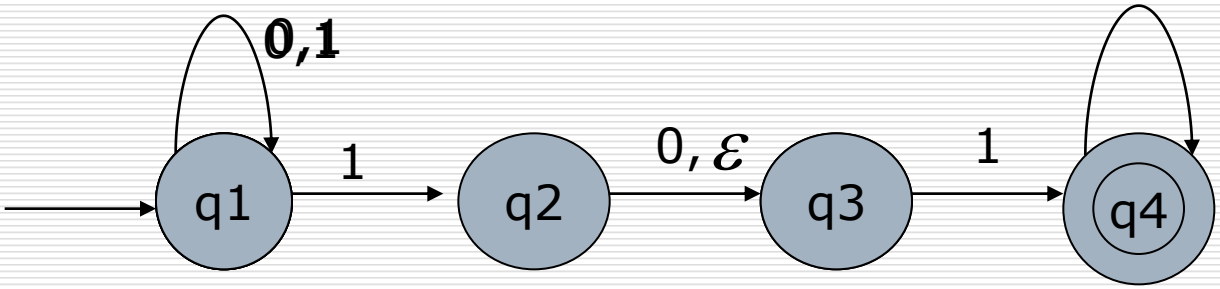
Design Automata

Suppose $\Sigma = \{0,1\}$ Construct a finite automaton M recognizing the following languages:

- A is the set of all strings with an odd number of 1's.
- A is the set of all strings 001 as a substring
- A is the set of all strings except 11 and 111
- The set of all strings except the empty string

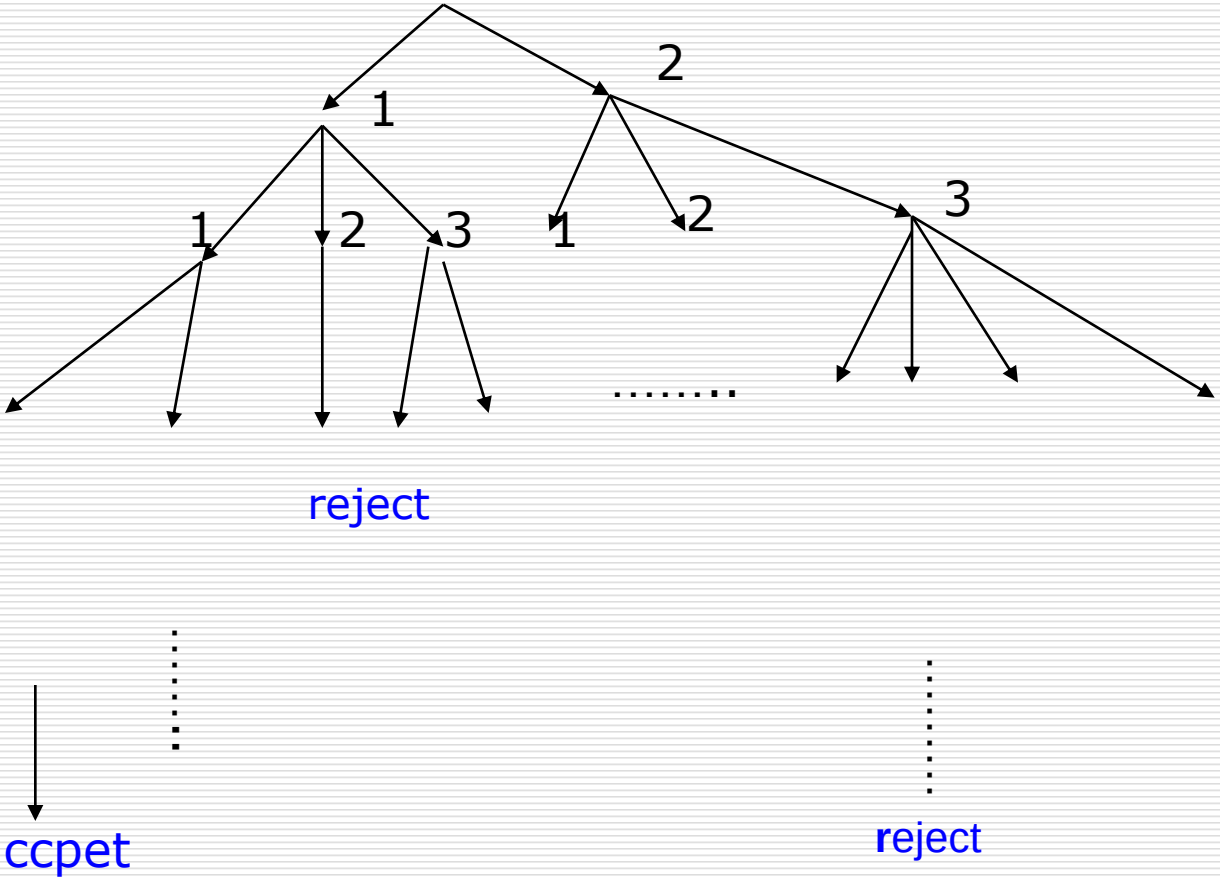
1. How do you know which one among equivalent automata is the minimal or the optimal?
2. Is there an algorithm for designing automata?

A glimpse of nondeterminism



Consider the computation of this FA on input 010110

Computation tree of nondeterministic FA

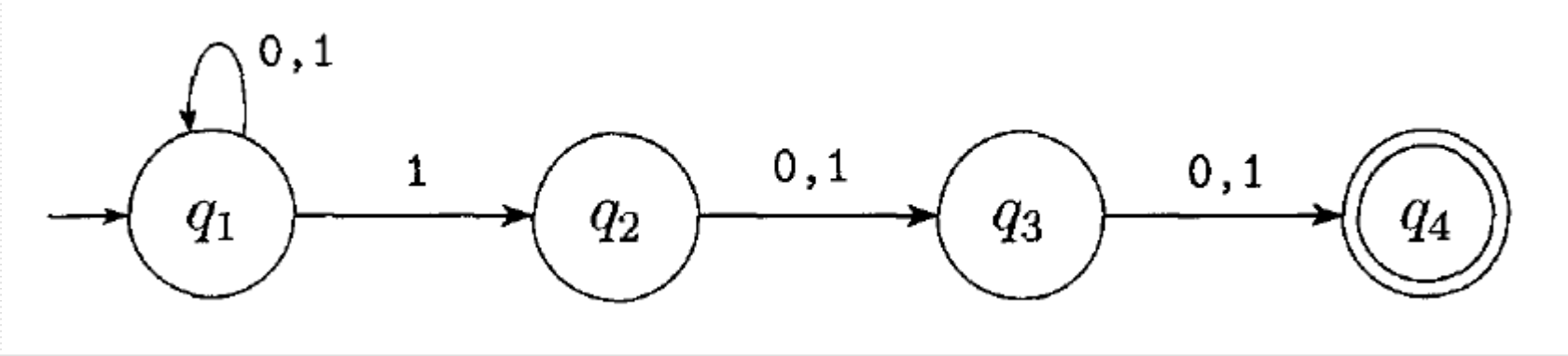


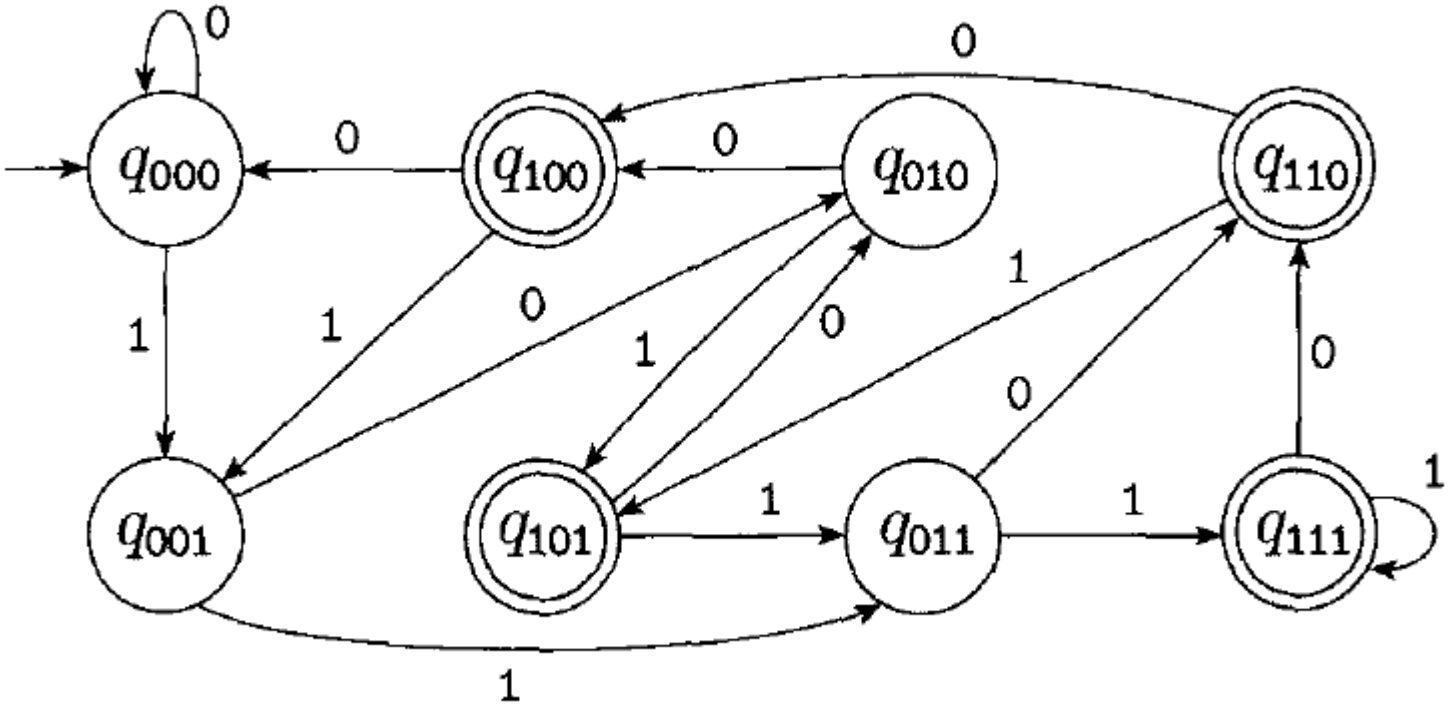
Designing NFAs

- Design a NFA that recognizes the language consisting of all strings over {0, 1} containing a 1 in the third position from the end.

Design an equivalent DFA and compare these two automata.

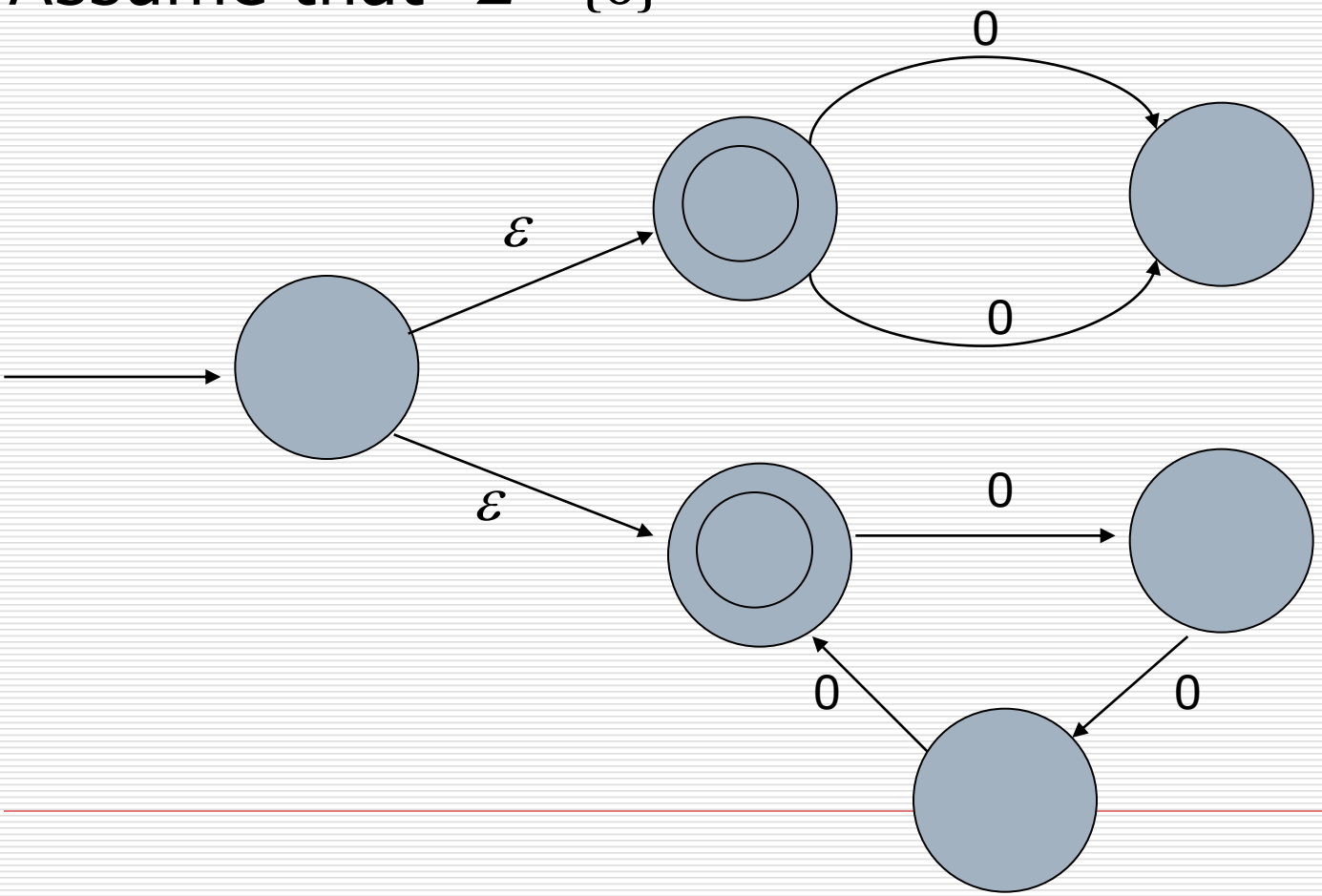
Comparison





Epsilon-transition

Assume that $\Sigma = \{0\}$



Nondeterministic Finite Automaton

A nondeterministic finite automaton is a 5 tuple $(Q, \Sigma, \delta, q_0, F)$

1. Q is a finite set called the states;
2. Σ is a finite set called the alphabet;
3. $\delta : Q \times \Sigma_\varepsilon \to 2^Q$ is the transition function;
4. q_0 is the start state, and
5. $F \subseteq Q$ is the set of accept states.

Two extensions: nondeterminism + the empty-string

Formal Definition of Computation

- A NFA M accepts input w if w can be written as $w = w_1 w_2 \cdots w_m$ where $w_i \in \Sigma_\varepsilon$ and sequences of states $r_0, \cdots, r_m \in Q$ exist that satisfy the following conditions:

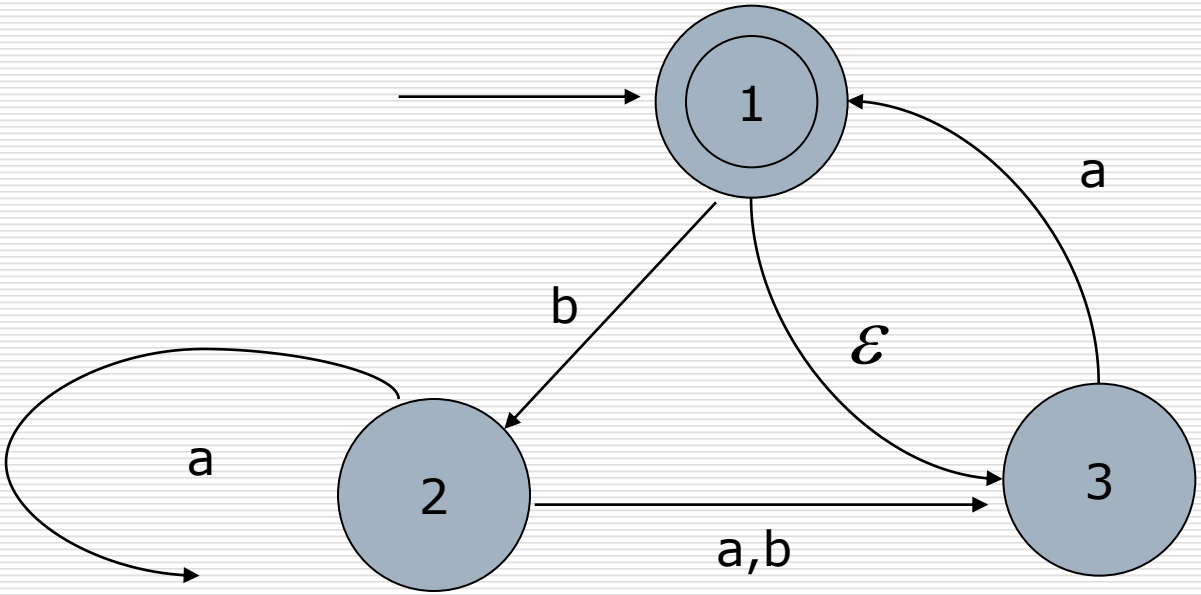
1. $r_0 = q_0$
2. For $i = 0, \cdots, m-1$, we have $r_{i+1} \in \delta(r_i, w_{i+1})$
3. rm is an accept state.

Equivalence of NFAs and DFAs

- Every nondeterministic finite automaton has an equivalent deterministic finite automaton.
- Proof: Subset construction by Scott and Rabin(1959).

Converting NFAs to DFAs

Assume that $\Sigma = \{a, b\}$



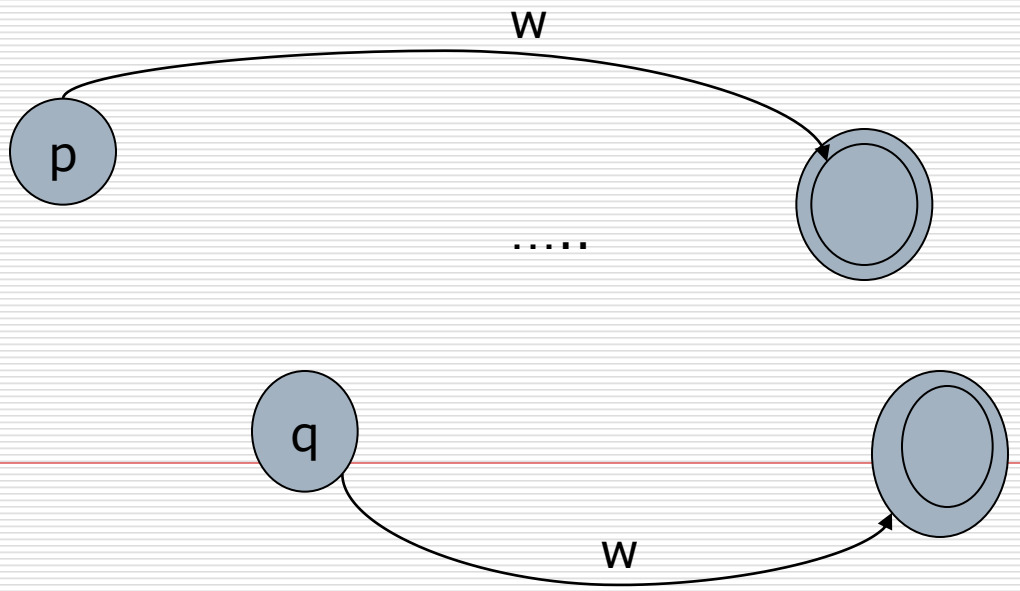
What is the expressive power of the NFA without epsilon transition?

Summary

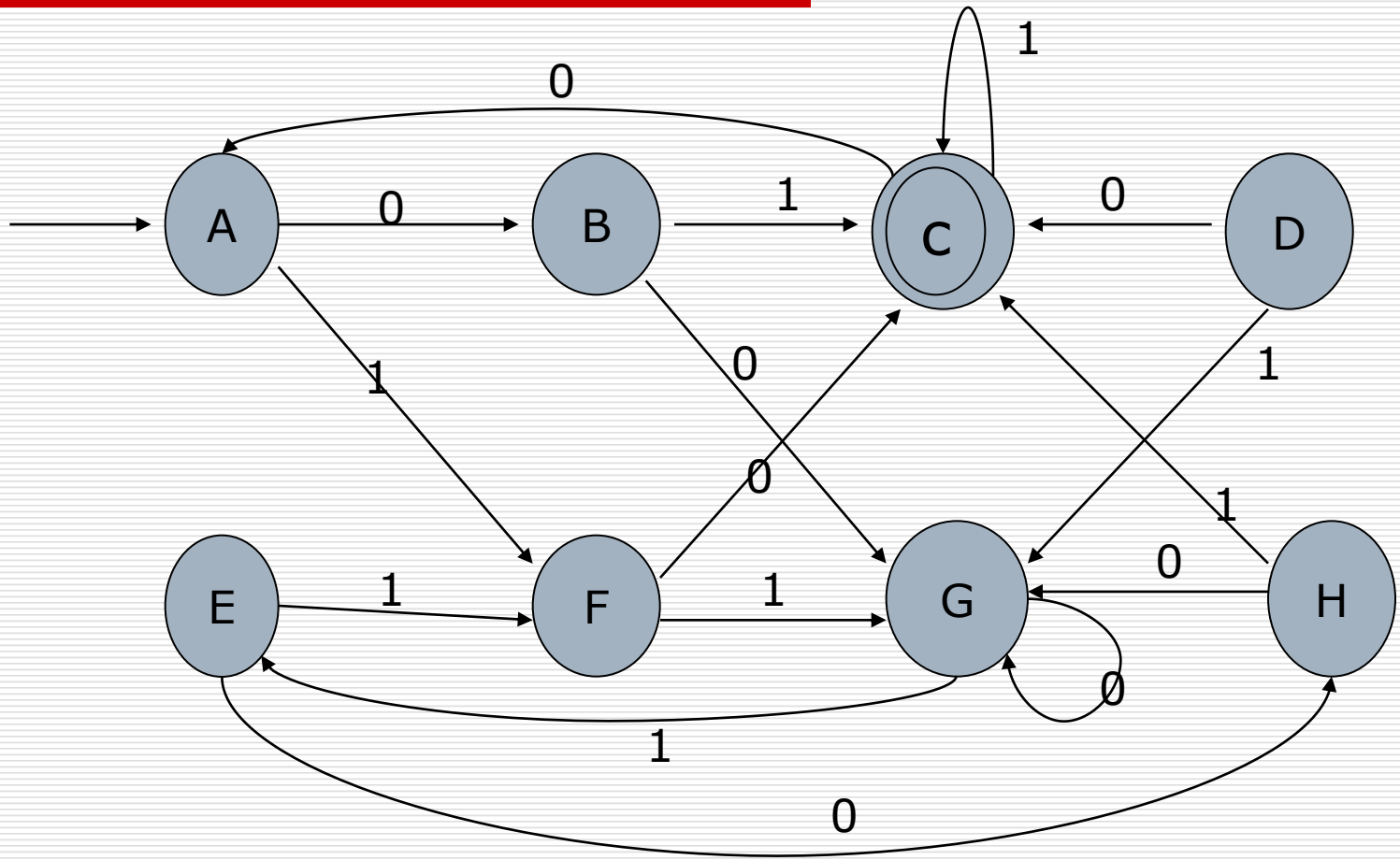
- DFA's, NFA's both accept exactly the same set of languages: the regular languages (next time).
- The NFA types are easier to design and may have exponentially fewer states than a DFA. Nondeterminism is a synonyms for guess or search.
- But only a DFA can be implemented!

Equivalence

- Given a DFA, two states p and q are equivalent if
- For all input strings w, $\hat{\delta}(p,w)$ is an accepting state iff $\hat{\delta}(q,w)$ is an accepting state.



Example



Distinguishable states

- Base case: If p is an accepting state and q is a nonaccepting, then the pair {p,q} is distinguishable;
- Induction case: Let p and q be states such that for some input symbol a, $r = \delta(p,a)$ and $s = \delta(q,a)$ are a pair of distinguishable states. Then {p, q} is a pair of distinguishable states.

Are accepting states indistinguishable?

Table-filling algorithm (Where should we start?)

	A	B	C	D	E	F	G	H
A								
B	x							
C	x	x						
D	x	x	x					
E		x	x	x				
F	x	x	x		x			
G	x	x	x	x	x	x		
H	x		x	x	x	x	x	

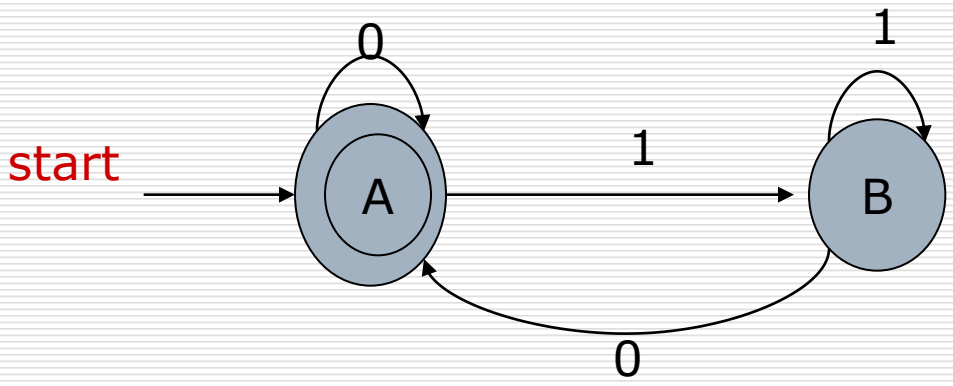
Why does this algorithm give the minimal automata?

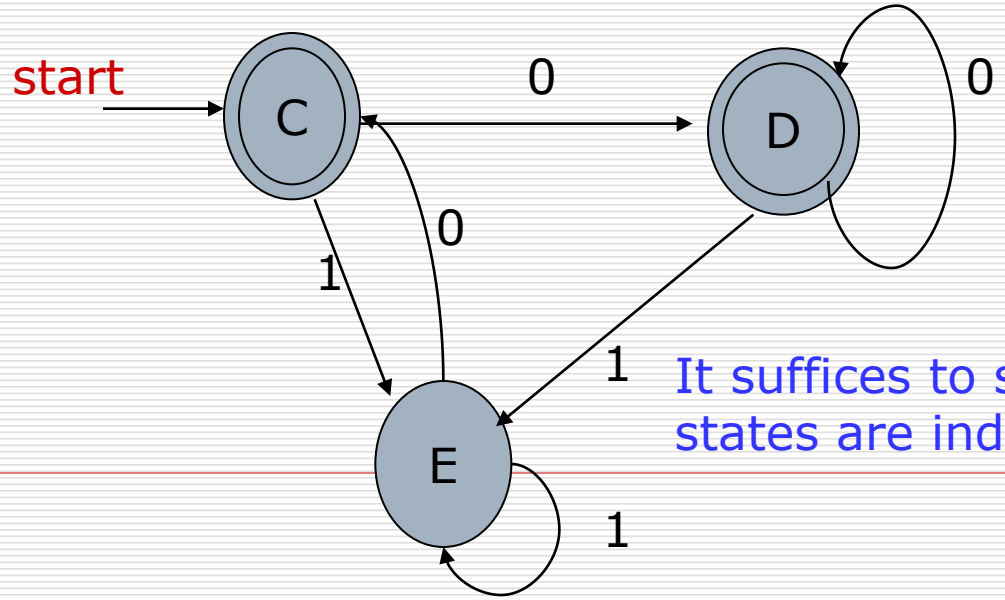
Testing equivalence of DFAs

- Recall that
  - two DFAs are equivalent if they accept the same language.

- Testing equivalence of DFAs can be transformed into testing equivalence of their starting states.

Example: Two equivalent DFAs





It suffices to show that the starting states are indistinguishable.